Lossless Text Compression Using PPM and Arithmetic Coding

Zhicheng Li

1 Overview

We compress `austen.txt` using order-5 prediction by partial matching (PPM), update exclusion, tuned probability weights, and 64-bit integer arithmetic coding. The compressed data and compiled decoder together occupy 23.3358% of the original size, with exact reconstruction.

2 Context Modeling and Probability Estimation

Prediction by partial matching (PPM) provides probability estimates. An arithmetic coder uses these estimates to produce a compressed bitstream. PPM estimates the probability of the next symbol using previously observed symbols as a context. When a longer context cannot predict the next symbol, the model backs off to a shorter one [1].

2.1 Context Representation

Our implementation reads the input file as a sequence of bytes, treating each byte as a symbol from the alphabet $\mathcal{B} = \{0, \ldots, 255\}$. Let $x_t \in \mathcal{B}$ denote the t-th input symbol, starting at $t = 1$. The *history* h stores the most recent symbols already processed. We retain at most $K = 5$ symbols, so $|h| = \min(K, t-1)$ before processing x_t. An order-k context s_k consists of the last k symbols of h, where $0 \le k \le |h|$. The order-0 context is the empty string ϵ. Thus, K is also the maximum context order. For each context s, a dictionary stores positive integer counts $c_s(x)$ for the symbols that have been inserted into that context. Define

$$\mathcal{A}_s = \{x : c_s(x) > 0\}, \qquad U_s = |\mathcal{A}_s|. \tag{1}$$

An absent context has an empty table. Contexts are stored as byte-string keys mapping to symbol-to-count dictionaries.

Both encoder and decoder start with empty dictionaries and empty history. They construct each prediction table before updating the PPM model with the current symbol. A *prediction table* lists the available symbols x and their integer weights $w_s(x)$ for context s. These weights are computed from the stored counts and determine the probability estimates used by the arithmetic coder. The update rule is described in Section 2.3.

2.2 Context Backoff and Symbol Exclusion

To construct prediction tables, we first search the available contexts. The PPM model visits $s_{|h|}, s_{|h|-1}, \ldots, s_0$ from longest to shortest.

For each new target symbol, initialize an exclusion set $E = \varnothing$. At the current order-k context $s = s_k$, the available candidates are

$$\mathcal{V}_s = \mathcal{A}_s \setminus E. \tag{2}$$

Here, $\setminus$ denotes set difference: remove all symbols in E from $\mathcal{A}_s$. If the target belongs to $\mathcal{V}_s$, encode it using the current context's prediction table and finish the search.

If $\mathcal{V}_s$ is empty, skip the context without coding an event. This covers both missing contexts and tables whose symbols have all been excluded.

If $\mathcal{V}_s$ is nonempty but does not contain the target, encode a special escape event ESC. Each nonempty prediction table contains both $\mathcal{V}_s$ and ESC. Internally, ESC is represented by integer 256. It produces no output symbol. After encoding ESC, set $E \leftarrow E \cup \mathcal{A}_s$ and continue to the next shorter context. This backoff mechanism allows the model to encode symbols not yet observed in the current context [1, 2].

Excluding higher-order predictions from lower-order tables is called *symbol exclusion* [2]. After an escape, these symbols cannot be the target, so their probability mass can be removed from subsequent symbol predictions. Symbol exclusion changes the prediction table without modifying the stored counts.

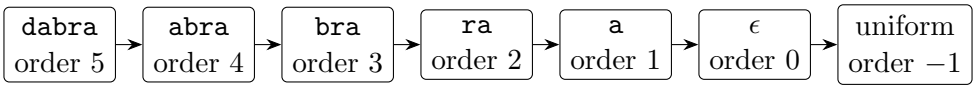


Figure 1: Search order for history `dabra`. Follow an arrow after an escape or an empty candidate set; stop when the target symbol is encoded. Each shorter nonnegative-order context removes the preceding context's first symbol.

If all nonnegative orders fail to find the target, use the order--1 table. It assigns integer

weight 1 to every symbol in $\mathcal{B} \setminus E$, with no escape event:

$$P_{-1}(x \mid E) = \frac{1}{256 - |E|}, \qquad x \in \mathcal{B} \setminus E. \tag{3}$$

Thus a previously unseen symbol can always be encoded. In particular, the first input symbol is encoded using the uniform distribution over all 256 symbols.

2.3 Model Update

After the arithmetic coder encodes or decodes the input symbol x_t, visit the contexts from longest to shortest. If a context s does not contain x_t, set $c_s(x_t) = 1$ and continue to the next shorter context. Create an empty table first if the context is missing. At the first context that already contains x_t, increment its count by 1 and stop. All shorter contexts remain unchanged.

This rule is called *update exclusion* [2]. It determines which stored counts change, whereas symbol exclusion determines which symbols are available during prediction. Consequently, the stored counts need not equal substring occurrence counts.

Finally, append x_t to the history and retain only its last K symbols. Both counts and history are updated once per input symbol. No updates occur between successive escapes for the same symbol.

2.4 Tuned Integer Weights

Before symbol exclusion, standard PPM-D assigns integer weights $2c_s(x) - 1$ to known symbols and U_s to escape [3]. Inspired by this estimator, we instead use:

$$w_s(x) = 200c_s(x) \quad (x \in \mathcal{V}_s), \qquad w_s(\text{ESC}) = 86U_s. \tag{4}$$

The constants 200 and 86 were chosen experimentally for this implementation; their selection is evaluated in Section 5.

For each nonempty candidate set, the normalizing total and probabilities are

$$T_s(E) = 200 \sum_{x \in \mathcal{V}_s} c_s(x) + 86U_s, \tag{5}$$

$$P_s(x \mid E) = \frac{200c_s(x)}{T_s(E)}, \qquad P_s(\text{ESC} \mid E) = \frac{86U_s}{T_s(E)}. \tag{6}$$

Importantly, U_s counts *all* distinct symbols stored in the context, including excluded ones.

Only the symbol term in $T_s(E)$ uses the reduced candidate set. The implementation does not replace U_s with $|\mathcal{V}_s|$.

Available symbols are ordered by increasing numeric value, followed by ESC; the order--1 table uses the same symbol ordering. The arithmetic coder uses the weights and cumulative sums directly. The decoder reconstructs the same tables and updates only after recovering a symbol, so frequency tables are not transmitted.

2.5 Worked Example

Consider encoding a space after `abracadabra`. Table 1 shows the state produced by the actual order-5 implementation with update exclusion. Symbols are displayed as ASCII characters. The history is `dabra`, giving the search order in Figure 1.

Order	Context	Stored counts	Excluded E	Available weights	Action
5	`dabra`	none	$\varnothing$	none	skip
4	`abra`	$c:1$	$\varnothing$	$c:200,\ \text{ESC}:86$	ESC
3	`bra`	$c:1$	$\{c\}$	none	skip
2	`ra`	$c:1$	$\{c\}$	none	skip
1	`a`	$b:2,\ c:1,\ d:1$	$\{c\}$	$b:400,\ d:200,$ ESC : 258	ESC
0	ϵ	$a:4,\ b:1,\ c:1,$ $d:1,\ r:1$	$\{b,c,d\}$	$a:800,\ r:200,$ ESC : 430	ESC
-1	default	not stored	$\{a,b,c,d,r\}$	251 remaining symbols, each with weight 1	space

Table 1: Model trace for a space after `abracadabra`. Exclusion sets are shown on entry to each row; skipped contexts produce no coding event.

Order 5 is missing. The order-4 escape excludes `c`, leaving no candidates at orders 3 and 2. At order 1, the remaining symbol weights are 400 and 200, but the escape weight is $86 \times 3 = 258$: the excluded `c` still contributes to U_s. The next escape also excludes `b` and `d`. Order 0 retains only `a` and `r`, while its escape weight uses all five stored symbols.

The event sequence supplied to the arithmetic coder is therefore

$$\mathrm{ESC}_4, \quad \mathrm{ESC}_1, \quad \mathrm{ESC}_0, \quad \mathrm{space}_{-1}, \tag{7}$$

where subscripts indicate context order. Its model probability, before integer interval rounding, is

$$P(\text{space} \mid \texttt{abracadabra}) = \frac{86}{286}\,\frac{258}{858}\,\frac{430}{1430}\,\frac{1}{251}. \tag{8}$$

Only after this complete sequence do encoder and decoder update their counts and history with the space. This illustrates how a single input symbol may require several coding events while still producing exactly one output symbol.

3 Integer Arithmetic Coding

The PPM model supplies a prediction table for each coding event. The arithmetic coder uses its integer weights to narrow a numeric interval. As the interval narrows, the coder outputs bits that identify the selected interval. We use the integer coding framework described by Witten, Neal, and Cleary [4] and Howard and Vitter [3]. The rules below specify our implementation, including its 64-bit state and bitstream termination.

3.1 Prediction Tables and Coding State

Let $e_0, \ldots, e_{m-1}$ be the m entries of the current prediction table, in encoding order. Each entry is an available input symbol or ESC. Let w_j denote the integer weight of entry e_j. For a nonnegative-order context s, this is $w_s(e_j)$ from Equation (4). At order -1, every entry has weight 1. Define the cumulative weights and total weight as

$$C_0 = 0, \qquad C_j = \sum_{i=0}^{j-1} w_i \quad (1 \le j \le m), \qquad T = C_m. \tag{9}$$

Thus, entry e_j has cumulative range $[C_j, C_{j+1})$ and probability estimate w_j/T. The ordering is the one defined in the preceding section: increasing symbol values, followed by ESC when present.

The coder maintains an integer interval $[L, H]$, including both endpoints. We define

$$F = 2^{64}, \qquad M = F/2, \qquad Q = F/4, \qquad L = 0, \quad H = F - 1. \tag{10}$$

Here, F is the size of the full range, M its midpoint, and Q one quarter of its size. The

interval is initialized once per file. It is carried across all coding events, including escapes and changes of context. Python integers allow intermediate widths and products to exceed 64 bits without overflow. All divisions below use integer arithmetic.

3.2 Interval Update

To encode entry e_j, first compute the current interval width $R = H - L + 1$. Then select its subinterval:

$$L' = L + \left\lfloor \frac{RC_j}{T} \right\rfloor, \tag{11}$$

$$H' = L + \left\lfloor \frac{RC_{j+1}}{T} \right\rfloor - 1. \tag{12}$$

Both expressions use the old value of L. Replace (L, H) with (L', H') only after computing both endpoints. The floor operation $\lfloor z \rfloor$ rounds z down to an integer. Larger weights receive wider subintervals, up to integer rounding.

3.3 Renormalization and Bit Output

After an interval update, the coder checks the three cases in Table 2. This step is called *renormalization*. It lets the coder output known bits while keeping the interval within the fixed range.

Some bits must wait until a preceding bit is known. Let p count these *pending bits*, with $p = 0$ initially. Define emit(b) for a bit $b \in \{0, 1\}$ as follows: output b, then output p copies of $1 - b$, and reset p to zero. For example, if $p = 0$, then emit(0) outputs `0`; if $p = 2$, then emit(0) outputs `011`.

Case	Interval condition	Offset δ	Encoder action
E1	$H < M$	0	emit(0)
E2	$L \geq M$	M	emit(1)
E3	$Q \leq L < M \leq H < 3Q$	Q	Increase p by 1; output no bit yet

Table 2: Renormalization rules. Subtract the offset δ from each endpoint before shifting left by one bit. Apply the rules repeatedly after every coding event.

For the selected case, perform its encoder action. Let $z \ll 1$ denote a one-bit left shift of

a nonnegative integer z. Update the endpoints as follows:

$$L \leftarrow (L - \delta) \ll 1, \qquad H \leftarrow \big((H - \delta) \ll 1\big) + 1. \tag{13}$$

The left shift appends a 0 to L. For H, the additional $+1$ appends a 1 instead, preserving the inclusive upper endpoint. Check the conditions again using the new endpoints. Stop renormalizing when none applies, then process the next coding event. In E1 and E2, the interval determines the next output bit. In E3, that bit is still uncertain. Each E3 step records a following bit that will be the complement of the eventual output bit. The pending count is retained across coding events until an emit operation resolves it.

3.4 Final Bit Emission

After all input symbols have been encoded, the encoder must finish the remaining interval. For nonempty input, first increase p by 1. Then apply

$$\begin{cases} \text{emit}(0), & L < Q, \\ \text{emit}(1), & L \geq Q. \end{cases} \tag{14}$$

This final emission selects a bit prefix for the remaining interval and resolves the pending bits.

3.5 Arithmetic Decoding

The arithmetic decoder starts with the same interval $[0, F - 1]$. It also maintains a *code value* V. Initialize $V = 0$, then read the first 64 bits of the payload in order. For each bit b, set $V \leftarrow (V \ll 1) + b$. Payload bit order and zero-padding are specified in Section 4.

For the current prediction table, compute its cumulative weights and $R = H - L + 1$. Map V into the cumulative-weight range using

$$u = \left\lfloor \frac{(V - L + 1)T - 1}{R} \right\rfloor. \tag{15}$$

Choose the unique entry e_j satisfying $C_j \leq u < C_{j+1}$. Use that entry to update the interval with Equations (11)–(12). The decoder then applies the same renormalization cases as the encoder. For each case, use its offset δ in Equation (13) and update the code value as

$$V \leftarrow \big((V - \delta) \ll 1\big) + b, \tag{16}$$

where b is the next payload bit. Repeat until no case applies. The decoder reads bits rather than emitting them; it does not maintain a pending-bit count. The recovered event e_j is returned to the PPM model. The file-level loop that requests these events is described next.

4 Compressed File Format and Reconstruction

The file contains the original byte count N followed by an arithmetic-coded payload. Both encoder and decoder use $K = 5$, the same 64-bit arithmetic state and weights, and initially empty context statistics.

Byte offset	Field	Representation
0–3	Original length N	Unsigned 32-bit integer, little-endian
4 onward	Arithmetic-coded payload	Bits packed most significant bit first

Table 3: Compressed file layout. The header occupies exactly four bytes. The payload size depends on the input and the model's predictions.

Payload bits are packed most significant bit first, with zeros filling any unused low bits of the final byte. The decoder reads from byte offset 4 in the same order; requests beyond the payload return zero, including when initializing the 64-bit code value. It stops after recovering exactly N bytes; escapes produce no output and do not advance this count. Empty input is represented by four zero header bytes and no payload. No end-of-file event is encoded.

For each input byte, the encoder processes the backoff tables until the byte is encoded, then updates the model once. After the last byte, it emits the final bits as specified in Section 3.4 and writes the header and packed payload to `compressed`. The decoder follows the same table sequence until a non-escape symbol is recovered, writes that byte to `out.txt`, and updates the model. Both sides retain their arithmetic state across all events and use binary file input and output.

5 Experiments and Results

5.1 Experimental Setup

All methods use the same `austen.txt` input of $N = 2{,}664{,}245$ bytes. Let C denote the complete compressed-file size and D the compiled-decoder size. The evaluation metric is

$$R_{\text{total}} = \frac{C + D}{N} \times 100\%. \tag{17}$$

All decoders were minified with python-minifier 3.2.0 and compiled at optimization level 0 with the embedded filename `decompress.py`. Measurements used an Apple M3 running macOS 26.7.1 and Python 3.12.4; reported times are median user CPU times from three runs of the compiled decoder, measured with `/usr/bin/time -p`. All 20 configurations reproduced the input byte for byte in all three runs. Parameter tuning uses this same input, so the selected settings are specific to this file.

5.2 Parameter Selection

Table 4 varies the context order while fixing the tuned weights, update exclusion, arithmetic coder, and four-byte header. Order 5 produces the smallest file among the tested orders, saving 23,411 bytes over order 4 and 3,648 bytes over order 6.

Maximum order K	Compressed size C (bytes)
4	640,200
5 (selected)	616,789
6	620,437

Table 4: Context-order comparison. All three decoders occupy 4,934 bytes.

Figure 2 compares nine escape factors between 80 and 110, with the symbol factor fixed at 200 and $K = 5$. The selected factor $\beta = 86$ gives the smallest file, but saves only one byte over 87. Relative to the untuned weights $2c_s(x)$ and U_s, tuning saves 422 compressed bytes under the same header; this is a small additional improvement.

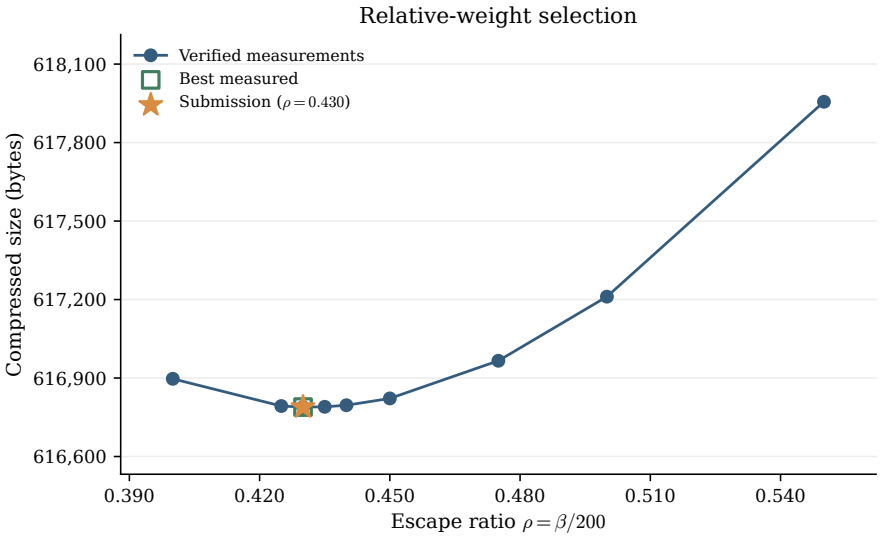


Figure 2: Escape-weight tuning at $K = 5$, with $\rho = \beta/200$. The smallest file occurs at $\beta = 86$ ($\rho = 0.43$).

5.3 Comparison with Other Compression Algorithms

The baselines are two-byte Huffman coding [5], static arithmetic coding [4], LZ78 [6], and LZW [7]. Their compressed sizes include the Huffman tree, static frequency table, or dictionary-code headers as applicable. LZ78 and LZW use fixed whole-byte index widths.

Method	Compressed bytes C	Decoder bytes D	R_{total}	Decode CPU time (s)
Huffman (two-byte groups)	1,308,812	3,300	49.2489%	0.49
Context-free arithmetic coding*	1,444,892	2,844	54.3394%	4.28
LZ78	1,475,432	1,414	55.4321%	0.10
LZW	1,247,197	1,578	46.8716%	0.15
Final tuned PPM + UE + arithmetic	616,789	4,934	23.3358%	7.73

Table 5: Size and decoding user CPU time (median of three runs). The starred baseline imports `bisect`, which is not permitted for submission.

The final method has the smallest total size among these implementations, using 50.21% less storage than LZW, the next smallest baseline. Its median decoding time is 7.73 seconds (range 7.62–7.91 seconds), slower than all four baselines. These results apply to this input and the tested implementations. The final decoder imports no modules; runtime validation on the required ECE Linux host remains separate from these local measurements.

6 CONCLUSIONS

On the project input, the selected order-5 PPM model with update exclusion, weights $200c_s(x)$ and $86U_s$, and 64-bit integer arithmetic coding produces a 616,789-byte compressed file and a 4,934-byte compiled decoder. Their combined size is 621,723 bytes, or 23.3358% of the original. All three measured decoding runs reconstruct the input exactly, with a median local user CPU time of 7.73 seconds. The parameter comparisons support the selected settings among the tested candidates. The final method uses the least storage among the tested implementations but decodes more slowly. These conclusions are specific to this input and local environment; ECE Linux runtime validation remains separate.

7 Acknowledgments

The main algorithmic ideas come from prior work: PPM and context backoff from Cleary and Witten [1], probability estimation and exclusion rules from Teahan [2], and integer arithmetic coding from Witten et al. [4] and Howard and Vitter [3]. Before developing the final method, I explored Huffman coding, arithmetic coding, LZ78, and LZW using code obtained from online sources or generated with AI assistance. The Huffman implementation was adapted from a GeeksforGeeks example [8]; the LZ78 and LZW implementations were adapted from Yousef Javaherian's LZW example on Stack Overflow [9]. I reviewed and modified these implementations, including testing two-byte symbols to improve Huffman compression.

After these trials, I asked AI for a suitable approach for this text. It recommended PPM with arithmetic coding and generated an initial PPM extension to the existing arithmetic coder, substantially improving compression. I then refined parameters and implementation structure using AI suggestions, online references, and my own reasoning, obtaining smaller additional gains. My contributions included reviewing the code, evaluating the alternatives, and refining the design; AI assisted with code generation and revision, as well as drafting and editing parts of this report.

References

[1] J. G. Cleary and I. H. Witten, "Data compression using adaptive coding and partial string matching," *IEEE Transactions on Communications*, vol. 32, no. 4, pp. 396–402, 1984. doi:10.1109/TCOM.1984.1096090.

[2] W. J. Teahan, “Probability estimation for PPM,” in *Proceedings of the New Zealand Computer Science Research Students Conference*, University of Waikato, Hamilton, New Zealand, pp. 267–274, 1995. Online text.

[3] P. G. Howard and J. S. Vitter, “Practical implementations of arithmetic coding,” in *Image and Text Compression*, J. A. Storer, Ed. Kluwer Academic Publishers, 1992, pp. 85–112. Technical report version.

[4] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, 1987. doi:10.1145/214762.214771.

[5] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952. doi:10.1109/JRPROC.1952.273898.

[6] J. Ziv and A. Lempel, “Compression of individual sequences via variable-rate coding,” *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 1978. doi:10.1109/TIT.1978.1055934.

[7] T. A. Welch, “A technique for high-performance data compression,” *Computer*, vol. 17, no. 6, pp. 8–19, 1984. doi:10.1109/MC.1984.1659158.

[8] GeeksforGeeks, “Huffman Coding in Python,” online code example. Source, accessed October 20, 2026.

[9] Y. Javaherian, Python LZW code example, Stack Overflow answer. Source, accessed October 20, 2026.

A Unminified Decoder Source

The submitted Python decoder is automatically minified using `python-minifier` to reduce its source size, which also reduces the compiled `.pyc` size in our implementation. For readability, this appendix presents the original, unminified decoder source.

Listing 1: My First Python Listing

```
STATE_BITS = 64
FULL_RANGE = 1 << STATE_BITS
HALF = FULL_RANGE >> 1
QUARTER = HALF >> 1
MASK = FULL_RANGE - 1
ESCAPE = 256

class PPMModel:
    def __init__(self):
```

```
        self.history = b""
        self.contexts = {}

    def distributions(self):
        # Start with the longest context, shortening it after each ESCAPE.
        excluded = set()
        for order in range(len(self.history), -1, -1):
            context = self.history[-order:] if order else b""
            counts_table = self.contexts.get(context, {})
            frequencies = [(symbol, 200 * count) for symbol, count in sorted(counts_table.items())
                           if symbol not in excluded]
            if not frequencies:
                continue
            # Match the encoder's 200 * count weights; ESCAPE gets 86 per
            # distinct symbol, including those removed by exclusion.
            frequencies.append((ESCAPE, 86 * len(counts_table)))
            yield frequencies
            excluded.update(counts_table)

        # Order -1: all remaining byte values have equal probability.
        yield [(symbol, 1) for symbol in range(256) if symbol not in excluded]

    def update(self, symbol):
        # Insert into escaped contexts, increment the first predicting one,
        # then exclude every lower order from this update.
        for order in range(len(self.history), -1, -1):
            context = self.history[-order:] if order else b""
            counts_table = self.contexts.setdefault(context, {})
            if symbol in counts_table:
                counts_table[symbol] += 1
                break
            counts_table[symbol] = 1
        self.history = (self.history + bytes([symbol]))[-5:]


def arithmetic_decompress(data):
    original_size = int.from_bytes(data[:4], "little")
```

```
47
48      bit_position = 4 * 8
49      bit_limit = len(data) * 8
50
51      def read_bit():
52          nonlocal bit_position
53          if bit_position < bit_limit:
54              bit = (data[bit_position // 8] >> (7 - bit_position % 8)) & 1
55          else:
56              bit = 0
57          bit_position += 1
58          return bit
59
60      low = 0
61      high = MASK
62      value = 0
63      for _ in range(STATE_BITS):
64          value = (value << 1) | read_bit()
65
66      def decode_symbol(frequencies):
67          nonlocal low, high, value
68          total = sum(count for _, count in frequencies)
69          width = high - low + 1
70          scaled = ((value - low + 1) * total - 1) // width
71          lower = 0
72          for symbol, count in frequencies:
73              if scaled < lower + count:
74                  break
75              lower += count
76
77          high = low + width * (lower + count) // total - 1
78          low = low + width * lower // total
79          while True:
80              if (high & HALF) == 0:
81                  pass
82              elif (low & HALF) != 0:
83                  low -= HALF
84                  high -= HALF
```

```
                value -= HALF
            elif (low & QUARTER) != 0 and (high & QUARTER) == 0:
                low -= QUARTER
                high -= QUARTER
                value -= QUARTER
            else:
                break
            low <<= 1
            high = (high << 1) | 1
            value = (value << 1) | read_bit()
        return symbol

    model = PPMModel()
    output = bytearray()
    for _ in range(original_size):
        for frequencies in model.distributions():
            symbol = decode_symbol(frequencies)
            if symbol != ESCAPE:
                break
        output.append(symbol)
        model.update(symbol)
    return output


with open("compressed", "rb") as fin, open("out.txt", "wb") as fout:
    fout.write(arithmetic_decompress(fin.read()))
```